

UNIVERSIDAD MIGUEL HERNÁNDEZ DE ELCHE
MÁSTER UNIVERSITARIO EN ROBÓTICA



REDES NEURONALES CONVOLUCIONALES PARA EL
ANÁLISIS DE SEÑALES EEG DURANTE LA MARCHA CON
UN EXOESQUELETO

TRABAJO FIN DE MÁSTER

Mayo - 2026

AUTOR: Daniel Rosique Egea

DIRECTOR: Eduardo Iañez Martinez

*“The human brain, sculpted by millions of years of evolution, is the sole architect
of the 'human universe’”*

- MIGUEL NICOLELIS

AGRADECIMIENTOS

Agradezco a ...

Gracias a ...

A ... por ...

RESUMEN

Se utilizarán redes neuronales convolucionales para la detección de patrones cerebrales asociados a la intención de iniciar o detener la marcha en sistemas de asistencia robótica. Este problema resulta clave en el desarrollo de interfaces cerebro-computador aplicadas al control de exoesqueletos, donde la identificación precisa de la intención del usuario es fundamental para garantizar un funcionamiento seguro y eficiente.

El objetivo de este trabajo es analizar el rendimiento de diferentes arquitecturas de redes neuronales convolucionales en la clasificación de señales EEG, así como evaluar distintas estrategias de entrenamiento, incluyendo modelos entrenados con datos del mismo día, enfoques acumulativos y técnicas de fine-tuning. Para ello, se emplean registros de EEG previamente adquiridos, aplicando esquemas de validación cruzada y evaluaciones en entornos pseudo-online que simulan condiciones reales de uso.

Los resultados obtenidos permiten comparar el impacto de la arquitectura, el método de entrenamiento y la variabilidad entre usuarios en el rendimiento del sistema, destacando aquellas configuraciones que maximizan la precisión y reducen los falsos positivos. Asimismo, se identifican diferencias significativas entre los distintos enfoques evaluados, lo que proporciona información relevante para el diseño de sistemas más robustos.

Finalmente, este trabajo sienta las bases para la aplicación de estos modelos en escenarios reales, contribuyendo al desarrollo de sistemas de control más fiables en exoesqueletos y otras tecnologías de asistencia basadas en interfaces cerebro-computador.

ÍNDICE

AGRADECIMIENTOS	III
RESUMEN	V
ÍNDICE DE TABLAS	XI
ÍNDICE DE FIGURAS	XII
ÍNDICE DE CÓDIGOS	XIII
1. Introducción	1
1.1. Motivación	1
1.2. Definición	2
1.3. Alternativas Consideradas	2
1.4. Objetivos a Conseguir	3
1.4.1. Objetivos específicos	3
2. Estado del Arte	5
2.1. Conceptos Relevantes del Dominio	5
2.2. Relación con Proyectos Similares	6
2.2.1. Métodos clásicos	6
2.2.2. Deep Learning aplicado a EEG	6
2.2.3. Aplicaciones en exoesqueletos / rehabilitación	6

2.2.4. Limitaciones actuales	7
2.2.5. Tu hueco (esto es clave)	7
3. Material y Métodos	8
3.1. Vision General del Sistema	8
3.2. Materiales	10
3.2.1. Hardware	10
3.2.2. Software	10
3.3. Estructura Dataset	12
3.3.1. Preprocesamiento	14
3.3.2. Preprocesamiento de datos	16
3.4. Modelos utilizados	16
3.4.1. EEGNet	17
3.4.2. DeepConvNet	17
3.4.3. ShallowConvNet	17
3.5. Metodología experimental	17
3.5.1. Variables del estudio	17
3.5.2. Estrategia de entrenamiento	17
3.5.3. Evaluación	18
3.6. Infraestructura Experimental	18
3.6.1. Diseño de experimentos	19

4. Resultados y Discusión	20
4.1. Tarea Estática	20
4.1.1. Comparación de Arquitecturas	20
4.1.2. Comparación de Método de Entrenamiento	20
4.1.3. Variabilidad Entre Usuarios	20
4.1.4. Mejor Configuración	20
4.2. Tarea Movimiento	20
4.2.1. Comparación de Arquitecturas	21
4.2.2. Comparación de Método de Entrenamiento	21
4.2.3. Variabilidad Entre Usuarios	21
4.2.4. Mejor Configuración	21
4.3. Hiperparametros	21
5. Conclusiones	22
BIBLIOGRAFÍA	23
ANEXOS	24
A. Primer anexo	24
B. Segundo anexo	25

ÍNDICE DE TABLAS

ÍNDICE DE FIGURAS

3.1. Pipeline experimental	9
3.2. Estructura temporal de un trial en términos de ventanas EEG.	13
3.3. Estructura jerárquica de los tensores de entrada y etiquetas utilizados en el modelo.	14

ÍNDICE DE CÓDIGOS

1. Introducción

Según la Organización Mundial de la Salud (OMS), las lesiones modulares afectan a más de 15 millones de personas a nivel mundial. Estas lesiones pueden ser ocasionadas por traumatismos o por causas no traumáticas, como tumores, enfermedades degenerativas y vasculares, infecciones o defectos congénitos. Las lesiones medulares producen una pérdida completa o parcial de las funciones sensoriales y/o motoras por debajo del nivel de la lesión y son una de las principales causas de discapacidad de larga duración [1].

Las barreras a la movilidad impiden a muchas personas participar plenamente en la sociedad, en los adultos se refleja con tasas de desempleo superiores al 60 % y en los niños una menor tasa de escolarización [1].

Las Interfaz Cerebro-computador (BCI) son una tecnología que se ha utilizado para ayudar a pacientes con problemas locomotrices, proporcionando un enfoque de rehabilitación basado en la interacción entre el cerebro y el cuerpo. El principal canal de comunicación analizado en estas interfaces es la señal eléctrica generada por el sistema nervioso central [2]. El uso de estas señales para controlar dispositivos es la mejor vía actualmente disponible para mejorar la calidad de vida de pacientes con lesiones modulares.

1.1. Motivación

La identificación precisa de patrones neuronales asociados a la intención de movimiento representa uno de los principales desafíos en el desarrollo de sistemas BCI aplicados a rehabilitación. La elevada variabilidad de las señales EEG, junto con su baja relación señal-ruido, dificulta la generalización de modelos entre sujetos y sesiones experimentales.

El desarrollo de técnicas basadas en redes neuronales convolucionales permite explorar nuevas estrategias para mejorar la detección de intención motora durante la marcha asistida con exoesqueletos, contribuyendo potencialmente al diseño de

sistemas de asistencia más adaptativos y eficientes.

1.2. Definición

- Qué señal tienes:
 - EEG
- Qué quieres detectar:
 - intención de caminar
 - parada
- Qué tipo de problema es:
 - clasificación binaria (o multiclase si aplica)
- Contexto:
 - entorno pseudo-online
 - datos offline

Muy importante: que quede claro qué entra y qué sale del sistema

Este trabajo aborda la clasificación de señales EEG registradas durante tareas de marcha con exoesqueleto, con el objetivo de detectar patrones cerebrales relacionados con la intención de caminar o detenerse.

Se pretende evaluar la capacidad de diferentes arquitecturas de redes neuronales convolucionales para extraer características discriminativas a partir de datos EEG preprocesados, analizando su rendimiento tanto en validación cruzada como en escenarios pseudo-online.

1.3. Alternativas Consideradas

Aquí haces un mini “estado del arte light”

Qué meter:

- Métodos clásicos:
 - features + SVM / LDA
- Métodos deep learning:
 - CNNs (EEGNet, DeepConvNet...)
- Diferentes enfoques de entrenamiento:
 - por usuario
 - generalizados
 - fine-tuning

No te lées mucho, esto no es el estado del arte completo Termina con:

“En este trabajo se opta por...”

1.4. Objetivos a Conseguir

Aquí tienes que ser cristalino

Objetivo general (1 frase) Desarrollar/evaluar modelos CNN para detectar intención de marcha en EEG

1.4.1. Objetivos específicos

Algo tipo:

- Evaluar diferentes arquitecturas (EEGNet, DeepConvNet...)
- Comparar métodos de entrenamiento:
 - same-day

- acumulativo
- fine-tuning
- Analizar variabilidad entre usuarios
- Evaluar rendimiento en:
 - validación cruzada
 - pseudo-online
- Reducir falsos positivos

Esto suele gustar MUCHO a tribunales

2. Estado del Arte

2.1. Conceptos Relevantes del Dominio

EEG

- Qué es (muy breve)
- Qué mide (actividad eléctrica cerebral)
- Problemas:
 - ruido
 - baja resolución espacial
 - variabilidad entre sujetos

BCI

- Qué es
- Pipeline típico:
 - adquisición → preprocesado → clasificación → acción

Aquí puedes conectar con tu caso (exoesqueleto)

Intención de movimiento

- Qué es detectar intención
- Tipos de señales:
 - motor imagery
 - movimiento real
- Importancia en control de dispositivos

Deep Learning en EEG

- Por qué CNNs
- Ventajas vs métodos clásicos:
 - menos feature engineering
 - mejor captura de patrones espaciales/temporales

2.2. Relación con Proyectos Similares

Estructura muy clara:

2.2.1. Métodos clásicos

- LDA, SVM
- Features manuales
- Limitaciones

2.2.2. Deep Learning aplicado a EEG

CNNs (EEGNet, DeepConvNet...)

Qué han conseguido

Problemas abiertos:

generalización

dependencia del usuario

2.2.3. Aplicaciones en exoesqueletos / rehabilitación

Uso de BCI para control

Problema clave:

detección fiable de intención

2.2.4. Limitaciones actuales

Aquí es donde te preparas el terreno:

variabilidad entre sujetos

necesidad de recalibración

falsos positivos peligrosos

falta de evaluación en escenarios realistas

2.2.5. Tu hueco (esto es clave)

Termina con algo así:

“En este contexto, este trabajo se centra en...”

Aquí conectas directamente con tu TFM:

comparación de métodos de entrenamiento

evaluación pseudo-online

análisis sistemático

3. Material y Métodos

3.1. Vision General del Sistema

Se ha definido un pipeline de ejecución que permite realizar múltiples experimentos de forma iterativa y automatizada. Este pipeline está compuesto por distintos módulos encargados de gestionar el preprocesamiento de los datos, la configuración de los modelos, el entrenamiento y la evaluación de los mismos.

Con el objetivo de facilitar la reproducibilidad y la exploración sistemática de parámetros, el código se ha estructurado como un microframework que permite modificar de forma sencilla las variables de estudio, así como controlar la ejecución de los experimentos. Este enfoque ha resultado especialmente útil dado el elevado número de ejecuciones.

Para organizar la ejecución, se ha definido una estructura jerárquica en tres niveles:

- **Experimento:** nivel superior que agrupa el conjunto completo de configuraciones evaluadas. Un experimento consiste en la ejecución sistemática de múltiples combinaciones de parámetros.
- **Iteración:** cada una de las ejecuciones individuales del pipeline dentro de un experimento, asociada a una combinación específica de parámetros (arquitectura, método de entrenamiento y tipo de tarea).
- **Entrenamiento:** nivel más bajo de la jerarquía, correspondiente al entrenamiento y evaluación de un modelo para un usuario y sesión concretos dentro de una iteración.

De este modo, cada experimento está compuesto por múltiples iteraciones, y cada iteración incluye varios entrenamientos, uno por cada usuario y sesión del dataset. Los experimentos se definen mediante la combinación de distintos parámetros, entre los que destacan el tipo de tarea, la arquitectura del modelo y el método de entrenamiento. Para cada combinación se ejecuta una iteración independiente

En cada iteración, el sistema inicializa los parámetros de entrenamiento y carga el conjunto de datos correspondiente al tipo de tarea. A continuación, se ejecuta el método de entrenamiento seleccionado, en el que se itera sobre los distintos usuarios y sesiones disponibles en el dataset. Para cada caso, se inicializa el modelo, se realiza el entrenamiento y se evalúa su rendimiento.

Finalmente, los resultados obtenidos, junto con los parámetros utilizados, se almacenan de forma estructurada, permitiendo su posterior análisis.

Este proceso permite realizar un estudio posterior de los resultados obtenidos por cada combinación de parámetros definida.

La Figura 3.1 resume la organización jerárquica del pipeline experimental.

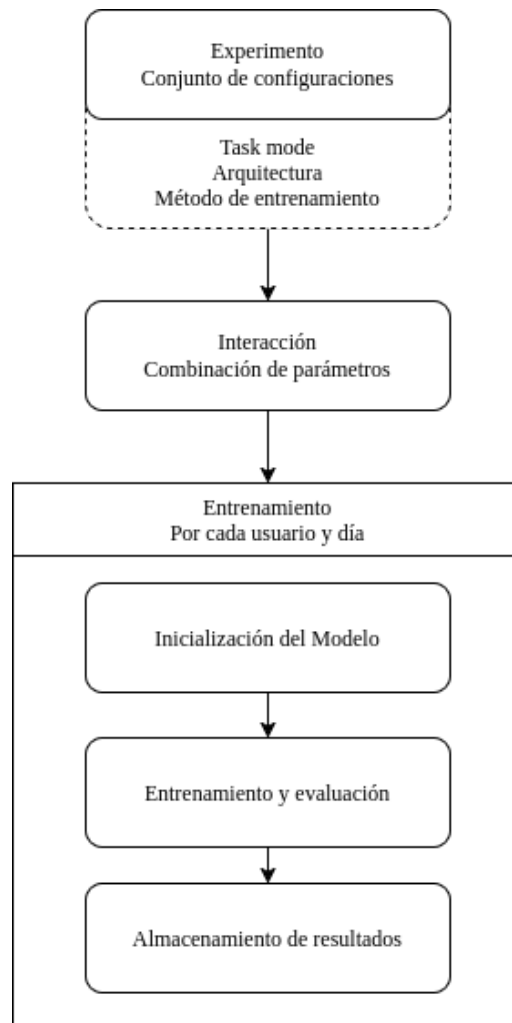


Figura 3.1: Pipeline experimental

3.2. Materiales

3.2.1. Hardware

Para ejecutar los distintos entrenamientos se ha utilizado un ordenador con las siguientes características:

- **CPU:** Intel Core i9-14900HX
- **RAM:** 31 GiB
- **GPU:** NVIDIA GeForce RTX 4060
- **GPU RAM:** 8 GiB

Estas características han resultado suficientes para completar los experimentos planteados sin limitaciones críticas. El tiempo medio de entrenamiento por iteración ha sido aproximadamente 40 minutos, dependiendo de la configuración del modelo y de los parámetros de entrenamiento.

El tiempo total de cómputo se estima mediante:

$$\text{Tiempo total (h)} = \frac{N_{\text{iteraciones}} \times t_{\text{iteración}}}{60}$$

Donde $N_{\text{iteraciones}} = 1917$ y $t_{\text{iteración}} = 40\text{minutos}$, resultando en un tiempo total aproximado de 766.8 horas ($\approx 31\text{días}$).

El uso de hardware con mayores prestaciones permitiría reducir significativamente el tiempo total de entrenamiento y facilitar la realización de un mayor número de experimentos.

3.2.2. Software

Para el desarrollo del proyecto se ha utilizado una infraestructura basada en herramientas ampliamente empleadas en el ámbito del aprendizaje automático y la

ingeniería de software.

Como lenguaje de programación principal se ha utilizado Python¹, debido a su amplio ecosistema de librerías científicas y de aprendizaje automático, así como su facilidad de integración con frameworks de alto nivel.

Para la construcción y entrenamiento de los modelos se han utilizado TensorFlow² y Keras³, frameworks de aprendizaje profundo que permiten definir, entrenar y evaluar redes neuronales de forma eficiente. Keras se ha empleado como interfaz de alto nivel sobre TensorFlow, facilitando la experimentación rápida con distintas arquitecturas.

Durante las etapas iniciales del proyecto se empleó Jupyter Notebook⁴ para la realización de pruebas exploratorias y la visualización preliminar de resultados. Sin embargo, su uso fue descartado en fases posteriores debido a la dificultad para mantener una estructura de código modular y escalable en experimentos de mayor complejidad.

Posteriormente, se adoptó Visual Studio Code⁵ como entorno de desarrollo principal, debido a su capacidad para gestionar múltiples tipos de archivos y su integración con herramientas de desarrollo. Se utilizaron extensiones específicas para Python, Jupyter y GitHub, permitiendo un flujo de trabajo unificado.

Como sistema de control de versiones se ha utilizado GitHub⁶, que facilita la gestión del código, el seguimiento de cambios y la colaboración con el tutor del proyecto. Además, permite la sincronización del repositorio entre distintos dispositivos de forma eficiente.

Para la visualización de resultados se ha utilizado Vega-Altair⁷, una librería de visualización estadística declarativa enfocada en la simplicidad, eficiencia y legibilidad. A diferencia de enfoques imperativos como Matplotlib, Altair permite definir visualizaciones de forma estructurada y reproducible, enfocándose en lo que se quiere

¹<https://www.python.org/>

²<https://www.tensorflow.org/>

³<https://keras.io/>

⁴<https://jupyter.org/>

⁵<https://code.visualstudio.com/>

⁶<https://github.com/>

⁷<https://altair-viz.github.io/>

observar en lugar de en los detalles de la implementación. Esto conduce a un código más limpio y legible[geeks'altair'2025].

Como gestor de base de datos se ha utilizado DuckDB⁸, un sistema de bases de datos analítico embebido que permite realizar consultas SQL de forma eficiente directamente sobre archivos locales. Su integración con Python y su alto rendimiento lo hacen especialmente adecuado para el procesamiento de grandes volúmenes de resultados experimentales[duckdb'2026].

Todas estas herramientas han sido integradas en un flujo de trabajo unificado orientado a la ejecución reproducible de experimentos.

3.3. Estructura Dataset

El dataset incluye registros de 5 sujetos, adquiridos a lo largo de 5 días diferentes (de lunes a viernes). Para cada día, se realizaron 11 ensayos (*trials*), y cada ensayo se segmentó en 56 ventanas temporales, lo que da lugar a un total de:

$$5 \times 5 \times 11 \times 56 = 15400 \text{ ventanas}$$

Cada *trial* corresponde a una secuencia estructurada de tareas mentales, compuesta por:

- 14 ventanas de relajación inicial,
- 28 ventanas de imaginación motora de la marcha,
- 14 ventanas finales de relajación.

Un esquema de esta segmentación temporal se muestra en la Figura 3.2.

⁸<https://duckdb.org/>

Relajación	Imaginación motora	Relajación
Ventanas 1–14	Ventanas 15–42	Ventanas 43–56
14 ventanas	28 ventanas	14 ventanas

Figura 3.2: Estructura temporal de un trial en términos de ventanas EEG.

El usuario alterna entre las dos tareas (static, y motion) durante los ensayos para que no sea tan repetitivo.

Cada instancia del dataset corresponde a una ventana de 400 muestras de señal EEG. Dado que la frecuencia de muestreo es de 200 Hz, cada ventana representa 2 s de señal. Las ventanas se organizan secuencialmente dentro de cada trial con un desplazamiento temporal de 0.5 s entre ventanas consecutivas.

Antes de cada tarea mental se le da una instrucción al usuario, esta instrucción puede alterar el clasificador, ya que no corresponde con ninguna tarea mental definida, por lo que se eliminan durante el preprocesamiento.

Adicionalmente, antes de cada ensayo se incluía un periodo inicial de duración aleatoria en el que el sujeto realizaba una tarea mental distinta. El objetivo de este periodo es el de reducir artefactos y ruido en la señal (convergencia de filtros). Este segmento fue eliminado durante el procesamiento previo del dataset.

Cada ventana de datos EEG está formada por 27 canales y 400 muestras por canal. Los 27 canales corresponden a los electrodos seleccionados para la adquisición de la señal. Por tanto, cada instancia del dataset puede interpretarse como una matriz de tamaño 27×400 , donde cada fila representa la evolución temporal de un canal EEG durante la ventana analizada.

Los 27 canales EEG utilizados fueron: F3, FZ, FC1, FCZ, C1, CZ, CP1, CPZ, FC5, FC3, C5, C3, CP5, CP3, P3, PZ, F4, FC2, FC4, FC6, C2, C4, CP2, CP4, C6, CP6 y P4. Estos electrodos se colocaron siguiendo el sistema internacional 10-10.

Los datos y las etiquetas están alineados a nivel de ventana, utilizando un fichero auxiliar que permite asociar cada segmento de señal con su correspondiente etiqueta.

3.3.1. Preprocesamiento

A partir de los datos disponibles, se ha realizado una reorganización estructural de los mismos con el objetivo de facilitar su uso en los modelos propuestos. En concreto, los datos se han reestructurado en tensores de seis dimensiones:

Input tensor shape: (5, 5, 11, 56, 27, 400)						
Dim.	1	2	3	4	5	6
Name	Users	Days	Trials	Windows	Channels	Samples
Size	5	5	11	56	27	400
Labels tensor shape: (5, 5, 11, 56, 1)						
Dim.	1	2	3	4	5	
Name	Users	Days	Trials	Windows	Label	
Size	5	5	11	56	1	

Figura 3.3: Estructura jerárquica de los tensores de entrada y etiquetas utilizados en el modelo.

Esta reorganización permite explotar explícitamente la estructura jerárquica del dataset durante el entrenamiento y la evaluación de los modelos.

En cuanto a la partición de los datos, se ha realizado una división entrenamiento-test con una proporción del 70 % y 30 %, respectivamente. Esta separación se ha llevado a cabo a nivel de *trial*, manteniendo la coherencia temporal de las ventanas.

La selección de los datos de test no se ha realizado de forma aleatoria, sino utilizando los últimos *trials* de cada sujeto y día. Esta decisión se basa en la hipótesis de que los sujetos presentan una mejora progresiva en el control de la imaginación motora a lo largo de la sesión, lo que permite evaluar el modelo en condiciones más estables.

Finalmente, no se han aplicado técnicas adicionales de preprocesamiento o filtrado sobre las señales EEG más allá de la estructuración descrita.

- adsad

¿Qué preprocesado EEG se aplicó?

filtrado (bandpass?) eliminación de artefactos (ICA?) normalización?

El dataset utilizado se compone por dos ficheros de labels y dos de datos de entrenamiento. Los labels contienen la intención de movimiento de cada canal de EEG, mientras que los datos contienen las lecturas de cada canal de EEG.

Estos ficheros están separados por la tarea a analizar, de esta forma tenemos los cuatro ficheros:

	Labels	Training
Static	labels_static_data.mat	training_static_data.mat
Motion	labels_motion_data.mat	training_motion_data.mat

- labels_static_data.mat
- training_static_data.mat
- labels_motion_data.mat
- training_motion_data.mat

Los ficheros contienen la siguiente estructura de datos: Los labels tienen la forma (15400, 1), mientras que los datos tienen la forma (15400, 27, 400). Los ficheros de labels contienen una matriz con el numero de ventanas y una columna con la intención de movimiento Los ficheros de training contienen una matriz de tamaño $N \times C \times S$, donde N es el número de ventanas, C es el número de canales y S es el número de muestras. En el caso de este experimento $N = 15400$ $C = 27$ y $S = 400$. Las muestras son obtenidas durante 2 segundos a 200hz

The 27 channels selected for acquisition were: F3, FZ, FC1, FCZ, C1, CZ, CP1, CPZ, FC5, FC3, C5, C3, CP5, CP3, P3, PZ, F4, FC2, FC4, FC6, C2, C4, CP2, CP4, C6, CP6, P4. They were placed following the 10-10 international system. Four electrodes were located next to the eyes to record electrooculography (EOG) and ground and reference electrodes were located on the right and left ear lobes, respectively. Each channel signal was amplified with BrainVision BrainAmp amplifier.

Se tienen los datos de 5 usuarios, con 5 días cada uno, 11 trials por día, cada día, 56 ventanas por trial. Por lo tanto, se tienen $5 \times 5 \times 11 \times 56 = 15400$ ventanas.

Cada trial corresponde a una secuencia de dos tareas mentales, relajación e imaginación de la marcha, que se repite en intervalos de 14 ventanas temporales ($0.5s \times \text{ventana} = 0.7s$) de relajación, 28 de imaginación de la marcha ($0.5s \times \text{ventana} = 1.4s$) y finalmente otras 14 de relajación. Se cuenta con un periodo inicial de duración aleatoria en la cual se le pedía al sujeto realizar una tarea mental compleja, diferente a la imaginación motora. Este periodo se utiliza para eliminar ruidos en la señal EEG y se ha eliminado del dataset final.

Un esquema de las ventanas se puede ver en la figura ??.

3.3.2. Preprocesamiento de datos

Los datos han sido procesados previamente en otro estudio.

La separación entre los datos de entrenamiento y testeo se ha hecho en un 70 % de entrenamiento y 30 % de testeo y se aplica al nivel de trial. Esta separación se ha hecho para cada entrenamiento del modelo y se toma en cuenta el usuario y día de cada uno. No se ha hecho de manera aleatoria, sino que se han utilizado para testeo empezando por el final de la lista de trials.

La razón de esta decisión es que en los primeros trials el sujeto suele presentar un menor control sobre su imaginación motora.

Más allá de estos preprocesamientos, no se ha hecho nada más

3.4. Modelos utilizados

Aquí metes:

3.4.1. EEGNet

3.4.2. DeepConvNet

3.4.3. ShallowConvNet

(si lo usas)

3.5. Metodología experimental

Aquí es donde entra todo tu proyecto de verdad.

3.5.1. Variables del estudio

Muy importante estructurarlo así:

Arquitectura del modelo

Método de entrenamiento:

same-day

acumulativo

fine-tuning

Tarea:

static / motion

Usuario

3.5.2. Estrategia de entrenamiento

epochs

batch size

callbacks:

early stopping

reduce LR

función de pérdida

métrica (accuracy)

No te lées demasiado, pero deja claro cómo entrenas

3.5.3. Evaluación

MUY importante:

validación cruzada

pseudo-online

explica qué significa pseudo-online (aunque sea breve)

3.6. Infraestructura Experimental

Aquí metes tu microframework (esto es TU aporte técnico fuerte)

sistema para ejecutar múltiples experimentos gestión de parámetros almacenamiento de resultados (DuckDB) automatización

Esto es diferencial → véndelo bien

Se ha programado un microframework para el desarrollo de experimentos de Machine Learning.

Como se ha comentado en el apartado ??, se ha realizado la comparación de tres modelos de redes neuronales convolucionales para la clasificación de la intención de

caminar o detenerse con un exoesqueleto. Se ha utilizado un dataset de EEG con 27 canales, obtenido de una muestra de un estudio previo XXXXXX. Los parametros a comparar son la arquitectura de la red, el metodo de entrenamiento, la tarea a predecir y la variabilidad entre usuarios.

Al necesitarse de cientos de ejecuciones para obtener todos los datos necesarios, durante las etapas iniciales se tomó la decisión de enfocar el proyecto de manera horizontal, preparando una base de ejecución sobre la cual modificar facilmente todo lo necesario para cada ejecución y permitiendo ejecutar multiples experimentos de manera secuencial.

(((((Aquí no sé si ir diicendo TOOOOODO lo que he ido poniendo en el código. Puedo ir siguiendo el timeline de Github y hacer un resumen de los commits))))))

3.6.1. Diseño de experimentos

Aquí explicas cómo organizaste TODO

combinaciones de parámetros nº de ejecuciones cómo comparas resultados

Esto conecta directamente con Results luego

4. Resultados y Discusión

se describirán y analizarán los resultados obtenidos siguiendo la metodología propuesta.

4.1. Tarea Estática

4.1.1. Comparación de Arquitecturas

Me da igual todo, solo me centro en los modelos. hago avg y std con los metodos y los usuarios

4.1.2. Comparación de Método de Entrenamiento

Me da igual todo, solo me centro en los metodos. hago avg y std con los modelos y los usuarios

4.1.3. Variabilidad Entre Usuarios

Me da igual todo, solo me centro en los usuarios. hago avg y std con los metodos y los modelos

4.1.4. Mejor Configuración

De aquí sacaría algo rollo: La mejor combinación en cuanto a usuario+metodo+modelo
-¿ej (gráfica solamente de usuario1+entrenamientoA+eegnet)

4.2. Tarea Movimiento

mismo esquema

4.2.1. Comparación de Arquitecturas

4.2.2. Comparación de Método de Entrenamiento

4.2.3. Variabilidad Entre Usuarios

4.2.4. Mejor Configuración

4.3. Hiperparametros

Analysis EEGNet hyperparameters comparison plots

5. Conclusiones

en funci3n de los resultados obtenidos, se expondr3an las conclusiones de la investigaci3n y se propondr3an trabajos futuros.

BIBLIOGRAFÍA

- [1] World Health Organization (WHO). *Lesión de la médula espinal*. Organización Mundial de la Salud. 16 de abr. de 2024. URL: <https://www.who.int/es/news-room/fact-sheets/detail/spinal-cord-injury> (visitado 29-04-2026).

- [2] M. T. Sadiq, M. Z. Aziz, A. Almogren, A. Yousaf, S. Siuly y A. U. Rehman. «Exploiting Pretrained CNN Models for the Development of an EEG-based Robust BCI Framework». En: *Computers in Biology and Medicine* 143 (1 de abr. de 2022), pág. 105242. ISSN: 0010-4825. DOI: 10.1016/j.combiomed.2022.105242. URL: <https://www.sciencedirect.com/science/article/pii/S0010482522000348> (visitado 27-04-2026).

ANEXOS

tantos como se considere necesario en funci3n de los aspectos concretos que se quieran especificar.

A. Primer anexo

Contenido del primer anexo (texto, tablas, figuras, c3digos, etc.)

B. Segundo anexo

Contenido del segundo anexo (texto, tablas, figuras, códigos, etc.)

interwordspace: 3.91663pt

interwordstretch: 1.95831pt

emergencystretch: 35.24963pt